

Experiments 3 – Bellman equations

NAME : Pranitha

REG NO : 192472073

COURSE CODE : MLA0305

COURSE NAME : Reinforcement learning

SLOT : B

Policy and Value Function Representation using Bellman Equations

Aim

To represent the policy and value function of an MDP using Bellman equations and find the best action for each state.

Procedure

1. Define the states, actions, rewards, and transitions.
2. Initialize the value of each state.
3. Apply the Bellman equation.
4. Update the value function repeatedly.
5. Compare action values for each state.
6. Select the action with the highest value.
7. Obtain the final optimal policy and value function.

Python code

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# User Input
# -----

gamma = float(input("Enter Discount Factor (0-1): "))
iterations = int(input("Enter Number of Iterations: "))

print("\nEnter Rewards for 3 States")
```

```

r1 = float(input("Reward for State S1: "))
r2 = float(input("Reward for State S2: "))
r3 = float(input("Reward for State S3: "))

rewards = np.array([r1, r2, r3])

states = ["S1", "S2", "S3"]
actions = ["Left", "Right"]

# Initial Value Function
V = np.zeros(3)

# Store values for graph
history = []

print("\nInitial Value Function")
print(V)

# -----
# Bellman Update
# -----

for i in range(iterations):

    newV = np.zeros(3)

    for s in range(3):

        left = rewards[s] + gamma * V[max(0, s-1)]
        right = rewards[s] + gamma * V[min(2, s+1)]

```

```
newV[s] = max(left,right)
```

```
V = newV.copy()
```

```
history.append(V.copy())
```

```
print("\nIteration",i+1)
```

```
print(V)
```

```
# -----
```

```
# Policy Representation
```

```
# -----
```

```
print("\n=====")
```

```
print("Optimal Policy")
```

```
print("=====")
```

```
policy=[]
```

```
for s in range(3):
```

```
    left = rewards[s] + gamma * V[max(0,s-1)]
```

```
    right = rewards[s] + gamma * V[min(2,s+1)]
```

```
    if left>right:
```

```
        best="Left"
```

```
    elif right>left:
```

```
        best="Right"
```

```

else:

    best="Stay"

policy.append(best)

print(states[s]," --> ",best)

# -----
# Final Value Function
# -----

print("\n=====")
print("Final Value Function")
print("=====")

for i in range(3):

    print(states[i],":",round(V[i],2))

# -----
# Summary Table
# -----

print("\n=====")
print("Summary")
print("=====")

print("State\tReward\tValue\tPolicy")

for i in range(3):

```

```

print(states[i], "\t", rewards[i], "\t", round(V[i], 2), "\t", policy[i])

# -----
# Visualization
# -----

history=np.array(history)

plt.figure(figsize=(8,5))

plt.plot(range(1, iterations+1), history[:,0], marker='o', label='State S1')
plt.plot(range(1, iterations+1), history[:,1], marker='s', label='State S2')
plt.plot(range(1, iterations+1), history[:,2], marker='^', label='State S3')

plt.title("Bellman Value Function Convergence")
plt.xlabel("Iterations")
plt.ylabel("Value Function")
plt.grid(True)
plt.legend()
plt.show()

# -----
# Policy Visualization
# -----

plt.figure(figsize=(6,4))

plt.bar(states, V)

```

```
plt.title("Final State Values")

plt.xlabel("States")

plt.ylabel("Value")

for i in range(3):

    plt.text(i,V[i],policy[i],ha='center')

plt.show()
```

Result

The value function for each state was calculated using the Bellman equation, and the optimal policy was obtained by selecting the action with the highest expected value.

Summary out put

```
=====
Optimal Policy
=====
S1 --> Right
S2 --> Right
S3 --> Right

=====
Final Value Function
=====
S1 : 106.26
S2 : 120.26
S3 : 130.26

=====
Summary
=====
State   Reward  Value  Policy
S1      5.0    106.26  Right
S2     10.0    120.26  Right
S3     20.0    130.26  Right
```

Visualization output

